Journal of Parallel and Distributed Computing 158 (2021) 115–125

Contents lists available at ScienceDirect

Journal of Parallel and Distributed Computing

www.elsevier.com/locate/jpdc

Teaching and learning HPC through serious games ☆

Julia Mullen [a,*], Lauren Milechin [b], Dennis Milechin [c]

[a] MIT Lincoln Laboratory, 244 Wood Street, Lexington, MA, United States of America
[b] Massachusetts Institute of Technology, Cambridge, MA, United States of America
[c] Boston University, Boston, MA, United States of America

ARTICLE INFO

Article history:
Received 30 October 2020
Received in revised form 26 February 2021
Accepted 18 July 2021
Available online 4 August 2021

Keywords:
HPC education
Serious games
HPC training

ABSTRACT

Serious games provide pathways for learners to develop intuition about concepts that are new to them. Such games are especially valuable in an educational context because they engage students in active learning and provide a means for students to develop mental models from their experiences. Furthermore, for short term learning experiences, such as informal, ungraded workshops, they act as ice-breaker activities that help build a sense of community among disparate learners. Collaborative games also spawn peer-to-peer discussion. These student experiences generally lead to deeper questions and discussions that allow instructors to uncover more concepts and clarify misunderstandings, all of which reinforce learning. The authors have created and incorporated role playing games within informal High Performance Computing (HPC) courses to provide students with tangible hands-on experiences with HPC concepts. All of the games are designed so that learners work collaboratively and take on a role associated with an HPC system. Engaging in the role playing highlights the strengths, weaknesses, and challenges of HPC. In this article we describe the games, how they were integrated into HPC courses, both in-person and virtual, and the lessons learned by both students and instructors. Though played with professional and university learners, the games are easily generalized to other cohorts.

© 2021 Elsevier Inc. All rights reserved.

1. Introduction

The authors have decades of experience educating and training High Performance Computing (HPC) users. The majority of this training has been through informal workshops or short 2–3 day, non-credit courses. These workshops include an introduction to theoretical concepts including the fundamentals of supercomputing and parallel programming paradigms as well as more concrete skill-based content illustrating the specific details required to use local HPC centers. While significant resources exist for training, including several online repositories supported by professional organizations and HPC consortia in the United States and Europe, [1–4], a common thread in these resources is the tendency to either be very abstract or dive too deeply into details, including details that are specific to a given HPC System. As a result, workshop creators and facilitators around the world are left to create their own materials to fill the gaps.

Even with good learning design, the compressed time frame associated with workshops limits opportunities to revisit and reflect on the material, activities that students regularly engage in during term or semester based courses. The result is that learners, especially novices, develop a superficial understanding of the discipline and often grasp the first HPC strategy presented to them and are later unable to scale their application beyond the most basic implementation. This is a symptom of a common experience that facilitators encounter post workshop, where students who appeared to understand the concepts and materials while at the workshop, send questions a few weeks post-workshop indicating a lack of understanding of the same material. In the late 1880s Ebbinghaus observed this phenomenon in his memory studies and captured it in an exponential curve called the "Forgetting Curve" [5]. The challenge for workshop creators and facilitators is to develop learning experiences that engage students so that they develop an intuition about the key concepts explored during the workshop and retain the understanding post-workshop. Gamification is one of the rec-

☆ DISTRIBUTION STATEMENT A. Approved for public release. Distribution is unlimited. This material is based upon work supported by the United States Air Force under Air Force Contract No. FA8702-15-D-0001. Any opinions, findings, conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the United States Air Force. © 2020 Massachusetts Institute of Technology. Delivered to the U.S. Government with Unlimited Rights, as defined in DFARS Part 252.227-7013 or 7014 (Feb 2014). Notwithstanding any copyright notice, U.S. Government rights in this work are defined by DFARS 252.227-7013 or DFARS 252.227-7014 as detailed above. Use of this work other than as specifically authorized by the U.S. Government may violate any copyrights that exist in this work.

* Corresponding author.
E-mail address: jsm@ll.mit.edu (J. Mullen).

https://doi.org/10.1016/j.jpdc.2021.07.014
0743-7315/© 2021 Elsevier Inc. All rights reserved.

ommended techniques for overcoming the Forgetting Curve [6] and the authors theorize that using serious games in HPC education and training could increase student intuition around parallel and distributed computing.

The use of games in education has a long history and the term "serious game" was first used by Abt [7] to describe games designed for learning. In particular, Abt stated that serious games must have an educational purpose [7] and not be played primarily for entertainment. More recently "serious games" have come to mean digital games. The United Kingdom's (UK) Future Lab review of educational games [8] included serious games, simulations and virtual worlds and concluded that while there is no fixed definition there is a general a consensus that serious games contain the following elements:

- a learning objective, implicit or explicit,
- content that is integrated into the game, and
- assessment that is either integral to the game or occurs during moderator guided reflection and discussion.

Catalano et al. [9] describe the integration of serious games into classrooms using the following framework; a brief that describes the learning objectives, rules, and game objective, facilitated game play, and a debrief that focuses on reflection and discussion to revisit the learning objectives, answer questions, and share experiences and observations.

The debrief component of the above framework provides a rudimentary opportunity to gauge student learning, but is qualitative at best. It is not clear how best to evaluate the effectiveness of serious games in the classroom. Early serious games, particularly digital games, used behaviorist theory and included assessments to determine scores, ranking, and to gamify the learning. These games integrated the assessments, but as games advanced in order to support constructivist [10] and experiential learning [11] they became more open-ended and explicit assessment became more difficult. DeGloria et al. [10] suggest assessing games by capturing learning and gaming analytics during game play. While difficult for unplugged activities, for digital games this has the benefit of providing insight for future game improvement and the inclusion of adaptive techniques that have been shown to improve learning. In guided unplugged activities, instructors often provide feedback to clarify and reinforce learning, but without capturing the entire event through video or an independent observer, it is hard to quantify and assess student understanding.

In a 2017 systemic literature review of articles about serious games for computer science, Petri et al. [12] analyzed the current state of serious game evaluation and found that the vast majority, 74 of the 117 studies, provided qualitative descriptions based on student comments during and after game play. Petri et al. considered this strategy to be an ad-hoc evaluation. Another 17 studies used a non-experimental, case study approach where the evaluation was based on collecting learner perceptions via post-game surveys. Full experiments utilizing testing, surveying, and control groups to evaluate the effectiveness of the game on learning were rare. Petri et al. provide some discussion of the challenges of building full experimental studies, such as small cohort sizes, informal learning, and inability to create a control group. They suggest that the inability to overcome these challenges is what leads to such a broad range of evaluation strategies. Petri et al.'s conclusion aligns with the UK Future Lab's findings that there is currently no consensus on how experiential games ought to be assessed [8].

In the games discussed in this paper, the instructors used the framework outlined by DeGloria et al., starting with a set of key questions that directly correspond to the learning goals and a brief description of the game rules and goals. Following the actual game play, the workshop facilitators led a discussion of the key questions to assess learner understanding and clarify any misunderstandings. In the initial trials the assessment was ad hoc, using student responses from the post-game discussion but in the most recent instance post-game surveys were conducted to capture student learning and impressions.

Serious games tend to be created within specified domains [9]. Battistella and von Wangenheim's [13] systemic review of games for teaching computer science an found 107 games covering a broad array of computer science topics. However, HPC differs from computer science in that most HPC novices have a deep background in their domain science with minimal exposure to basic computing and system architecture concepts. For this reason, though HPC games may be applicable to computer science, rarely is the inverse true. A quick survey of the literature uncovers a connection between HPC, simulation, modeling and games [14]. However, the focus is on the use of HPC to support simulation and modeling for games rather than games designed for HPC education. In this context simulation is the result of an HPC program being executed to provide a virtual or digital game environment.

As part of their Education and Outreach efforts, the Edinburgh Parallel Computing Center (EPCC) at the University of Edinburgh, in the UK created a serious game specifically directed towards HPC concerns. The game objective is to help the player understand the trade-offs between the key decisions required to keep an HPC Center active and viable [15]. While this provides important insight into the complex process of managing and operating an HPC center and is vital for training HPC system professionals, the game is not as beneficial to researchers who develop or use HPC scientific and mathematical software. The typical attendee in an HPC workshop is a researcher who needs to use the system in order to scale up their application either to speed it up, handle large data sets or both. As such, these learners are focused on their research domain, how to quickly develop their application and how best to use the supercomputing system at their location. However, novice users, whether they are creating HPC software or using commercial software, need to understand some basic concepts about the challenges associated with distributed computing and the impact that HPC architectures have on the execution of their applications. Without understanding these basic concepts, researchers whose primary computing experience is limited to a laptop computer have difficulty scaling their applications, don't know how to request appropriate resources and are generally frustrated by the steep learning curve inherent in using a shared production HPC system. For these reasons, the authors were searching for games or classroom activities where the learning objectives aligned with these basic concepts.

The authors are aware of two unplugged classroom activities that involve students actively simulating an HPC concept or function. The first is the puzzle demonstration that Henry Neeman at the Oklahoma Supercomputing Center for Education and Research (OSCER) at the University of Oklahoma includes in his Supercomputing in Plain English course [16,17]. In the course, puzzles are used to illustrate how speed-up can be obtained by distributing tasks among a number of people. In Supercomputing in Plain English the discussion starts with a single person completing a puzzle, then two people, increasing the number of people working on the puzzle until everyone is vying for the puzzle pieces. At that point it becomes necessary to divide the pieces and people and place them at two tables. From there it is easy to build analogies to a number of HPC concepts including speed-up, resource contention, and the challenges of working with distributed data.

The second classroom activity, from Tim Warburton of Virginia Tech [18], is one he used to explain the challenges of communication in distributed computing. The students in the course simulated the All-to-All communication pattern. The activity was used

prior to the advent of multi-core machines, so data distribution was based on a flat distribution of processors. Each student was given a processor ID and a value and the students simulated the All-to-All by walking messages to other processors. The "Be the Processor" game described in Section 4 was motivated by memories of discussions about this simulation.

The games discussed here were developed for a physical classroom setting but contain the elements described previously and are thus considered serious games. All of the games were included in informal in-person workshops which did not include formal assessments. With the onset of the Covid19 pandemic, the authors were forced to address the challenges of developing learning activities that work in virtual environments. The original decision to develop games using tangible components for in-person activities rather than digital games came from the desire to remove abstractions that often obscure concepts for the novice learner. Watching a computer simulation of communication is the same as watching the results from a parallel simulation and lacks the experiential component that helps develop intuition. The in-person activities described here have the added benefit of serving as icebreakers when they bring students together to complete tasks or solve problems. The challenge is to build virtual versions that retain both the experiential and collaborative nature of the in-person games. The January 2021 version of the Practical HPC workshop included a blend of online, asynchronous, concept videos and live, virtual, meetings for games, discussions and computer simulation demonstrations. This blended approach required modifications to the game play. While none of the games was converted to a fully digital game, the "Be the Scheduler" and "Be the Processor" games were restructured for the live, virtual meetings. The conversion from in-person to live, virtual games are discussed the respective sections of the paper that present the games.

The paper is arranged as follows. Starting with Section 2, the "Puzzle Game", and moving through the "Be the Scheduler", Section 3, to "Be the Processor", Section 4, each game has a dedicated section to describe the game, the motivation behind it's development, how it was used in the local workshops and conversion from an in-person format with tangible items to a virtual format. After a discussion about the lessons learned by the instructors in Section 5, the paper briefly discusses next steps (Section 6) and closes with a summary, Section 7.

2. Example 1: the puzzle game

The Puzzle Game is a modification of the demonstration and discussion used by Henry Neeman in the introduction to his Supercomputing in Plain English [16] course, as described in the Introduction. In the Supercomputing in Plain English course, the primary learning objectives for the puzzle demonstration include the constraints on speed-up, resource contention, and the challenges of working with distributed data. While the authors have similar learning objectives for their Practical HPC (PHPC) course, they support a wider range of parallel workflows than the average HPC center. This broader application space requires greater emphasis on concurrent computing versus strictly parallel computing, where concurrent computing is an umbrella term that includes high throughput applications. With this broader range of workflows, students need to develop intuition about distributed data and learn to consider the key design questions:

- Where is my data?
- Where does my data need to be?
- Which programming paradigm best fits my application needs?

Through the game play, puzzles are used to illustrate how speed-up can be obtained by distributing tasks among a number of people. Since most students are familiar with puzzles, the task is well understood and allows instructors to make the analogy between a puzzle piece and a data element. Using this analogy it is fairly easy for students to think about data distribution and data flow patterns for their applications.

Table 1
Timings for the shared memory puzzle.

Team	Team Size	Time (mins)
Serial	1	7:00
Group A	3	7:33
Group B	3	6:15
Group C	2	6:04
Group D	2	4:55

Table 2
Timings for the distributed memory puzzle.

Distribution	Time (mins)
Serial	25:00
Block	8:20
Cyclic	17:15

2.1. Game objectives

The "Puzzzle Game" is a collaborative game where student teams complete puzzles in shared and distributed memory contexts. The learning objectives of the Puzzle Game include understanding

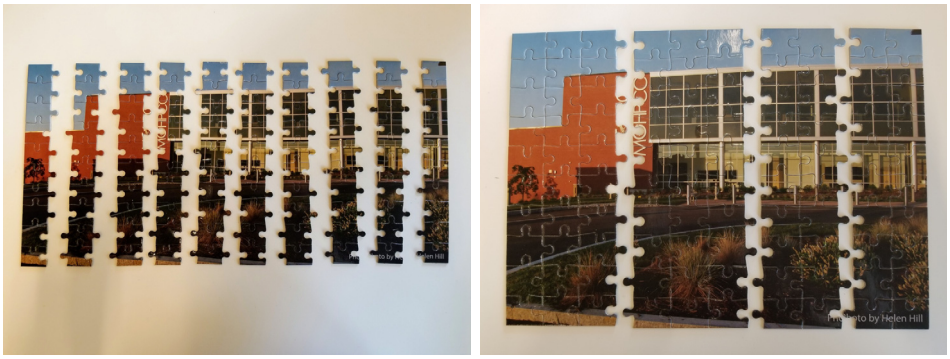
- data distributions,
- the effects of shared and distributed memory, and
- communication and communication latency.

The game objectives are to complete the puzzle, capture the timing results, and hopefully complete the puzzle in less time than the baseline serial results produced by the instructors.

The "Puzzle Game" has been used twice in informal classes. The puzzles used in the first run took too long to complete, and were replaced with smaller, less complex puzzles, particularly for the shared memory exercise. A 30-piece puzzle was used for the shared memory portion, and 100-piece for the distributed memory exercise.

2.2. Game play

Prior to the game the instructors complete 2 puzzles, capture the time and record it as the serial time to completion. The game starts with teams of two to three people at a table working together on a 30 piece puzzle. In this mode the table represents a compute node with shared memory space and two to three cores. Each person is a core and they work together to complete the puzzle, recording the time to completion. The timings for the "shared memory" puzzle completion are shown in Table 1. In the second activity a 100 piece puzzle is distributed into four parcels using either cyclic (Fig. 1a) or block (Fig. 1b) distributions. The students are placed in teams and each team is placed at a separate table. Each table represents a rank with its local memory space and two people complete the task of a single rank. Once situated the students are given a bag with the puzzle pieces and their Processor ID (MPI Rank) and are told to start their timer and complete the puzzle. The timings for the distributed puzzle completion are shown in Table 2.



(a) Cyclically distributed puzzle. (b) Block distributed puzzle.

Fig. 1. Puzzle distributions.

2.3. In the classroom

Following the completion of the puzzles, the guided discussion focused on questions posed at the start of the game:

- Is it better to have more people?
- Is it better to have fewer people?
- What were the challenges?
- Which distribution was better for this problem and why?
- Can you think of a problem where other distributions would be better?

Through the game students experienced why "more is not always better", a concept that instructors often have trouble conveying to novice parallel program developers. Students understood that there are multiple ways to split up data and tasks and explored when such splits were appropriate and useful. As a consequence of sending messages via "sneaker net" (walking the pieces to another table) some of the puzzle sections fell apart, representing lost data. This was particularly true for the cyclically distributed puzzle. Additionally, because it was harder to see the partial picture from the completed sections, the cyclically distributed puzzle took significantly longer than the block distributed version. While this is an artifact of using puzzles and "sneaker net", it prompted a discussion of delayed or missing messages and their affect on the overall completion of a job.

Finally, one puzzle had a hanging processor–one group took significantly longer to complete their portion of the puzzle. One student response was to go help students on the slower node, but the instructors pointed out that processors can't relocate from one node to another. This real-world scenario provided an opportunity to highlight communication latency, dynamic task scheduling, the effects of hung processors, and even touch upon the issues associated with load balancing. All of these topics were revisited throughout the remainder of the workshop most especially during sections on MPI and parallel programming.

2.4. Virtual adaptation

As the authors prepared for the January 2021 blended version of the PHPC course, they considered strategies for implementing the games into a virtual, live session. A search of digital puzzle applications revealed many free applications for collaborative, online, puzzle activities. Unfortunately, there was no way to distribute the puzzle pieces in a way that would mimic the data distributions associated with parallel computing. While completing a puzzle together online could demonstrate the speed-up achieved by distributing tasks, the other learning objectives were not supported and it was determined that converting the "Puzzle Game" was not viable at this time.

3. Example 2: be the scheduler

The "Be the Scheduler" game allows students to take on the role of the scheduler in an HPC system. This game was inspired by the question that HPC users often ask: "Why isn't my job running?". It was developed for a class aimed at current and near-future users of the HPC Systems on the MIT Campus. These systems tend to have very heterogeneous job types and workloads, and we wanted a game that would demonstrate how the scheduler handles the resource requirements of a variety of job types.

3.1. Game objectives

In the game, Lego Bricks represent the cluster, nodes, and different types of jobs, and cards represent submission scripts. Students draw cards and decide where the jobs should run and launch them by placing the job bricks onto the nodes. The goal of this game is to schedule and runs all the jobs in the job submission card deck.

The learning objectives are to:

- Understand the role of the scheduler
- Develop an intuition about how resources are allocated
- Review scheduler commands
- Understand the range of job types and resources required by each
- Understand the difference between distributed and shared memory

The job types that we refer to are those workloads that are commonly run on the HPC Systems on the MIT Campus, which are described in the next section. The game uses terminology that is specific to the system's scheduler, Slurm [19], both to help review scheduler commands and to familiarize students with that terminology.

3.2. Game play

To set up the game board, flat gray Lego bricks are placed on a flat board, as shown in Fig. 2. This represents the cluster, and the gray bricks represent the nodes in the cluster. The size of a node represents the number of cores on that node. For example, an 8×2 brick represents a node with sixteen cores. There are two node types: CPU nodes and GPU nodes. GPU nodes have flat 1×1 yellow bricks placed adjacent to the node to represent GPUs, as shown in the node on the bottom in Fig. 2.

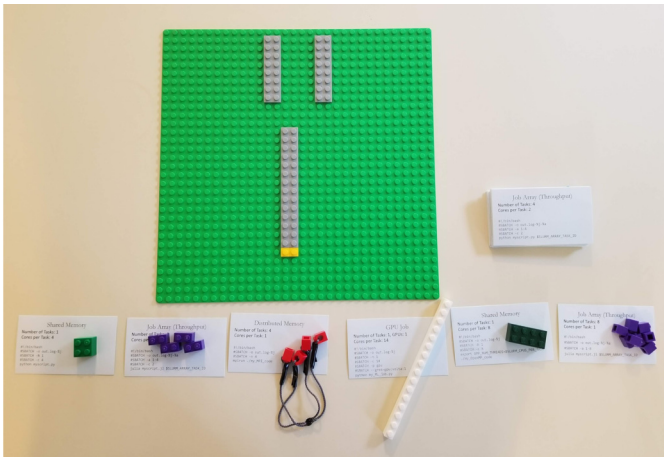


Fig. 2. Scheduler Board Setup. The large green board represents the cluster, the small flat gray blocks are nodes. The node on the bottom has two yellow 1x1 flat blocks, representing GPUs. Starting game cards and pieces are shown at the bottom, from left to right, a Shared Memory job, a Job Array, a Distributed Memory (MPI) job, a GPU job, another Shared Memory job, and another Job Array.

The board is placed on a table with space for two lines, or queues, of cards below it. These are the Running and Pending queues. The Running Queue contains the cards for jobs that are currently running, or placed on the board, and the Pending Queue contains jobs that are waiting to run, either due to unavailable resources or an unsatisfied dependency. Job cards are placed to the side of the board. As shown in Fig. 3, the front side of the card shows the type of job, the resources that are being requested, and a short submission script. The back of each card shows the job status output using the syntax of the scheduler.

During game setup, six cards are drawn and their requested jobs pieces are selected (shown in Fig. 2) and launched. Students launch a job by placing the bricks corresponding to that job on the board, and then placing the card face down in the Running Queue so the job status can be seen. Lego bricks of different colors and shapes represent different types of jobs, and are described below. This setup can be done collaboratively and does not have to be done in turns.

The game proceeds by taking turns going through the job card deck until all the jobs have completed. Fig. 4 shows a board during play with jobs running on the cluster and waiting in the Pending Queue. For each turn the student:

1. Cleans up the completed job: The completed job is determined by the roll of a die. The number on the die is the job that has completed, for example rolling a four means the fourth job in the Running Queue has completed. If there is no job in that spot, the student continues to the next step. To clean up the completed job, the player removes the corresponding pieces from the board, and places the card in the discard pile. The remaining cards are shifted to fill in the empty space.
2. Launches any pending jobs: First the student draws a card from the deck, places it at the end of the Pending Queue, and places the corresponding job pieces on the card. Next, starting at the beginning of the Pending Queue, they launch any jobs that can run. As jobs start, the player turns the card over, placing it at the end of the Running Queue, and places the job pieces on the board. The different job types are:
   - Job Array: These are High Throughput Jobs and are represented by groupings of purple 1×1 and 1×2 bricks, the 1×2 bricks are used in jobs that require two cores per task. The player selects the number and type of pieces matching the card and places them anywhere on the CPU nodes. These job tasks are independent, so do not need to be on the same node and not all pieces must be placed at once. Any remaining pieces are placed on the card and left in the Pending Queue.
   - MapReduce: These jobs consist of two steps, a Map step and Reduce or gather step. The job cards are held together with a paper clip. The Map step is scheduled like a Job Array. The Reduce step has a dependency on the Map step, and remains in the Pending Queue until the Map step has completed. The MIT cluster has specialized software for submitting these types of jobs (LLMapReduce [20]) and they are a common workflow. Many centers support similar jobs so they are included in the game.
   - Shared Memory or Large Memory: These are represented by single green $n \times m$ sized bricks of various sizes to represent the number of cores requested. The piece with the right number of "cores" is selected (for example, a four core job can be either a 2×2 or 1×4 brick) and placed on a node. Since the job requires that all processors see the same memory space, each piece must be placed fully on only one CPU node.
   - MPI/Distributed Memory: MPI and Distributed jobs are groups of 1×1 bricks that have been tied together with a string, representing the network. Individual tasks can be placed on different CPU nodes, as they are connected and can communicate over the network. These must all be placed on the board at the same time.
   - GPU: GPU jobs are represented by 1×16 bricks to occupy half of the GPU node. Each GPU node has two GPUs, and can run two GPU jobs at a time. Only GPU jobs should be placed on the GPU node(s).

Consider Fig. 4 as an example. For this turn a 2 has been rolled, indicating the job on the second card in the Running Queue at the bottom has completed. This Shared Memory job is represented by the piece on the node on the top right. The player removes the piece, moves its card to the discard pile, and moves the remaining Running Queue cards to the left to fill the gap. Another card is chosen from the top of the draw pile and placed at the end of the Pending Queue, on the right, and its corresponding pieces placed on the top of the card. Then, starting from the beginning of the Pending Queue (the Job Array job on the left), pieces would be placed on the board as resources are available. In this case, all jobs through the Map step of the MapReduce job will fit on the board.

The turn is over when there are no more pending jobs that can start. The students continue to take turns until all the jobs have started.

3.3. In the classroom

We have run this game twice in person during an annual Practical High Performance Computing (PHPC) class that is offered during MIT's Independent Activities Period (IAP) in January 2019 and 2020, and once as a demonstration during an Outreach and Education Birds of a Feather session at Supercomputing 2019. We have also run this game virtually once over Zoom during the PHPC IAP course in January 2021, which we discuss below. Before the game the instructors demonstrated a few rounds to show how the game is played. In the in-person PHPC class players were split into two groups of about six, including one instructor for each group to assist in game play and answer questions. Each group got their own board, set of cards, job pieces and die to play the game. The smaller teams meant no one had to wait too long for their turn. After the game the two groups reconvened and had a group discussion to debrief. In the second running of the course we prepared questions to guide discussion.

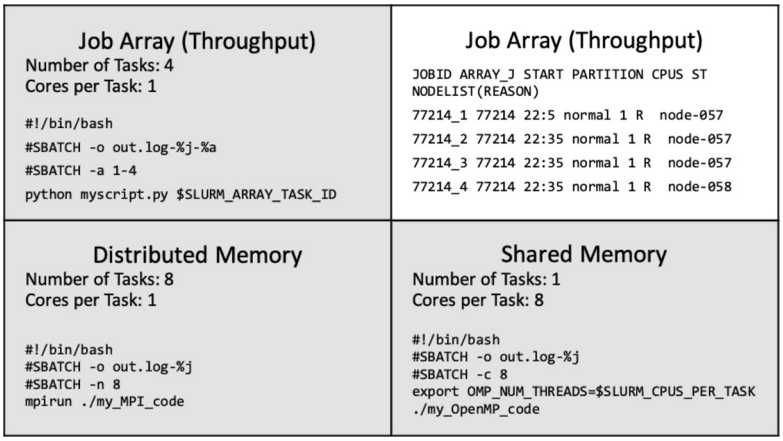


Fig. 3. Sample Job Cards. The front of the card shows the job type, requirements and submission script, the back shows the job status output. The front of the cards is gray-toned to more easily differentiate from the back.

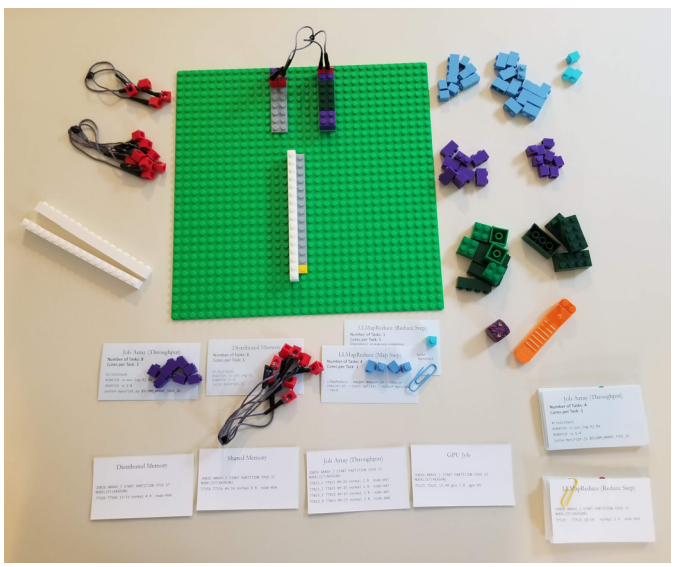


Fig. 4. Scheduler Layout Mid-Game. The board shows four jobs currently running, which correspond to the Running Queue, the line of cards at the bottom. Three jobs are pending in the Pending Queue, the line of cards above the Running Queue: a Job Array job, a Distributed job, and a MapReduce job. The game pieces are shown to the side of the board. On the left there are Distributed and GPU jobs, and on the right MapReduce, Job array, and Shared Memory (all listed top to bottom).

In the first iteration of the game the board was not pre-loaded with any jobs. Both groups found that the game was slow to start. The nodes took a long time to fill up, so jobs would run immediately without waiting in the Pending Queue. We trialed pre-loading the board during setup at the SC19 Birds of a Feather session and found this made for more interesting game play from the beginning.

While playing the game the students were able to see first hand why a given job didn't start, and how the scheduler handles different job types. In particular, they noticed that Shared Memory, Distributed, and the Reduce steps of MapReduce jobs sometimes waited several turns in the Pending Queue before starting. They were able to see the link between a Reduce step sitting in the Pending Queue because it had a dependency on a Map step versus a Shared Memory job waiting for available cores on the same node. Students also experienced the case where, even though the Distributed job was at the front of the Pending Queue, tasks from the Job Array behind it were starting to run because they could start independently, whereas tasks from the Distributed job had to start at the same time. The lesson here is not only why a particular job didn't start, but that they should put in the effort to find out exactly what their job requires and request only that, rather than requesting more than they need.

3.4. Virtual adaptation

Where in-person game play is not possible, the "Be the Scheduler" game can be played virtually over a video web-conference call. The facilitator has the game setup on their desk with a camera showing the board and cards in play. The game proceeds as it would in person, where players take turns identifying completed jobs and launching new jobs, with the facilitator removing and placing the physical pieces. The facilitator rolls the die and draws the cards, showing the results to the camera. A digital version of the game is also in development, which we will briefly discuss in Section 6.

We ran this game virtually during the PHPC IAP class in January 2021. The game was played in a single group. Because each turn took a bit longer than running in person, we played fewer rounds, where each participant took one turn. After a few turns the game play sped up, as the students grew more confident with the game. The format led to more discussion during the game, whereas previous years we had more post-game discussion.

Running the game through a camera introduced a challenge: it was more difficult for the participants to see and read the game cards. The facilitator addressed this by reading the cards as they were drawn in addition to showing them to the camera. In the past we added a few labels to make it easier to identify game pieces, but in this case the camera would have made hand-written labels difficult to read.

If we were to play the "Be the Scheduler" game over a video call in the future we would adjust the game cards so they are easier to read. We would also prepare labels for both the game pieces and nodes. Labeling the nodes would allow the players to more easily specify where a job should be scheduled. However, we expect that the digital version of the "Be the Scheduler" game discussed in Section 6 will be complete before the next virtual game play.

3.5. Evaluation

In the first two years of the PHPC class, post-course surveys focused on the learning objectives of the class and did not include any questions specific to the games. One student did state that "The interactive exercise was very helpful", which may refer to one of the games, and when asked "What are your top take-aways from this course?" one student wrote "Patience for the scheduler!", which seems to refer to the "Be the Scheduler" game.

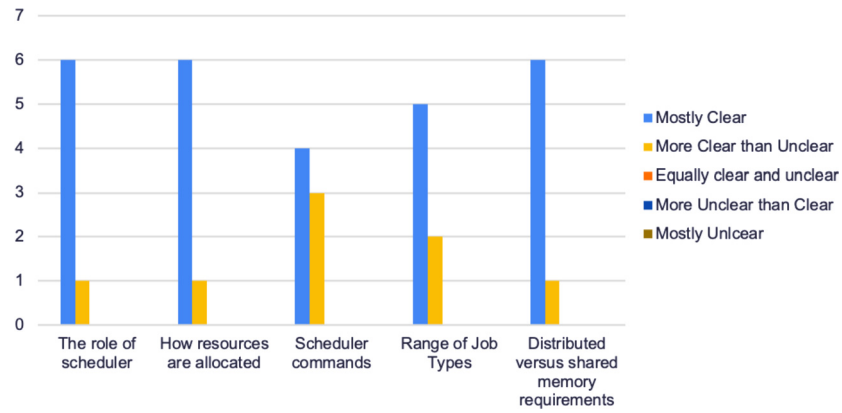


Fig. 5. Survey results for the "Be the Scheduler Game". Students were asked to say how clear the game made the listed topics.

Table 3
Survey results for the "Be the Scheduler Game". Students were asked to reflect on the "Be the Scheduler" game and state whether the game helped them know what to expect when submitting and monitoring their jobs.

True	False
6	1

We were able to run a game-specific survey after the virtual PHPC class taught in January 2021. Students were asked two questions related to the "Be the Scheduler" game. First, students were asked to state how clear the game made several topics, which were derived from the learning objectives. The second question asked whether the game helped them know what to expect when submitting and monitoring their jobs.

We received 7 responses to the survey, which is roughly half of the number of students who attended the entire class. These results are shown in Fig. 5 and Table 3. For the first question, all students stated that the game made each topic either mostly clear or more clear than unclear, with most students stating that it was mostly clear. Fewer students said the game made the scheduler commands mostly clear, which would be expected because the game did not have a heavy focus on the commands. The scheduler commands are also incorporated into the game through the job cards, which were difficult to see through the camera. In the second question, 6 out of 7 students said that the game helped them know what to expect when submitting or monitoring their jobs (Table 3). Overall, the surveyed students came away confident in their understanding of the scheduler.

4. Example 3: be the processor

As HPC architectures evolve, hybrid computing models join existing shared and distributed memory computing paradigms in order to match hierarchical hardware design. Inspired by Warburton's [18] All-to-All activity, described in the Introduction, the authors created a game where the participants collaborate to perform the actions of an HPC system to complete a task. The game was designed to help students develop intuition about creating applications for modern HPC systems. In 2016 this game was used in a stand-alone lecture about HPC systems that was part of a data science course. In 2021 a virtual version was used in the Practical HPC (PHPC) course. Both versions of the game are described in this section.

4.1. Game objectives

In this game, students take on the role of compute cores, performing tasks as part of a node within a larger cluster. The learning objectives, achieved through hands-on activities, include understanding:

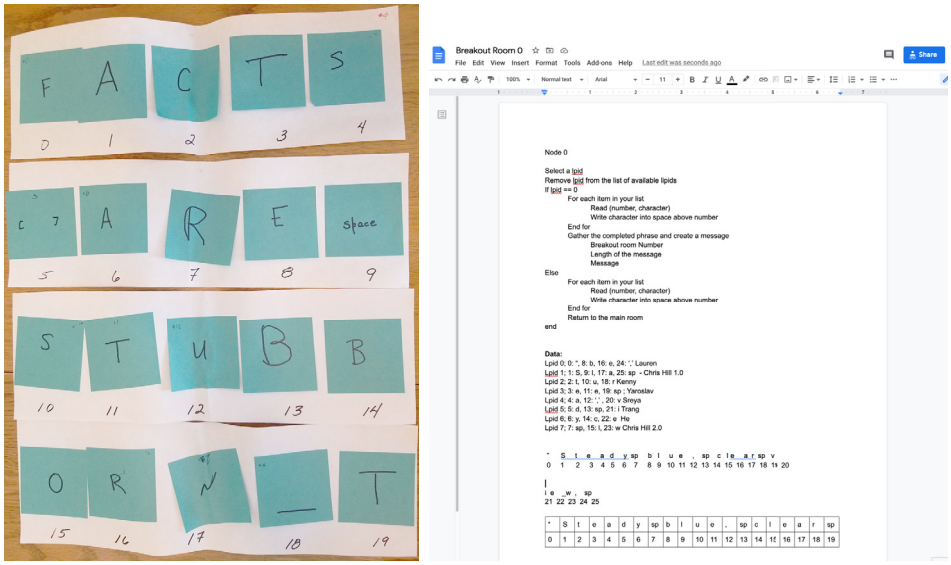
- shared versus distributed memory,
- communication and communication latency,
- load balancing,
- memory spaces, and
- data distributions.

The goal was to work together to create a phrase or sentence from individual data elements. In the 2016 in-person version of the game, the game was played with roughly 40 professionals and the message was a quote from John Adams. Any quote or statement would have worked, as long as it was of sufficient length to provide each student with roughly three instructions to complete while supporting six to seven "nodes", or groups of students.

4.2. Game play

The student activity mimics the work of a cluster of multi-core nodes. For the in-person version, the students were distributed into teams of six players each. The teams represented a node and members represented processors on the node. Each team was given a card with their node number within the class "world", a local memory space represented by a sheet of card stock marked with individual memory slots, and a stack of sticky notes to write the characters into memory. In addition, each participant was given instructions, a stack of index cards, and a number representing their MPI rank (global). Each index card had the data required for the task and the memory location where the output (character) was to be stored. For each node, the participants must decide who is the lead processor and that "core" is responsible for "sending" the local node message data to the global leader.

Play begins once the participants have been assigned to their nodes and received their packet of instructions and data. In the in-person 2016 version, the instructions were to read the number on an index card, convert it to a character based on a cipher table provided in the instructions, write the character on a sticky note and place the sticky note in the assigned memory slot (Fig. 6a). Each participant received three to four index cards, i.e. data elements, and completed these tasks at their own pace. Once this first step was complete, each team, or node, held a phrase on their memory cards. The second step was for each team to send their portion of the quote to global Rank 0, which was defined to be Node 0, Core



(a) Memory register for in-person game. (b) Memory register for virtual game.

Fig. 6. Memory registers for the "Be the Processor" game.

0. When Rank 0 had received all of the messages, the full message was posted. In practice, node leaders posted their completed message to a "buffer" location on the classroom wall that corresponded to their node. This modification was highlighted because the physical movement around the room provided a clearer representation of data movement between nodes and allowed learners to see communication latency, asynchronous messaging and messages arriving out of order. Additionally, posting to the wall made it easier for students to see the complete message.

4.3. In the classroom

The in-person game was played with computer science and engineering professionals who expected to use supercomputing resources for data science applications. The discussion directly after the game suggested that the game achieved many of the learning goals described earlier in this section.

During the post game discussion, the primary take-away was a more complete understanding of the requirements for message passing in parallel and distributed computing as well as some of the challenges associated with message passing. For example, there were a few cores who were noticeably slower at the task than others so the entire group had to wait for the last two processors to post their portion of the message. This led to a short discussion about asynchronous behavior and load balancing. Initially the tasks were distributed cyclically on the node and by block across nodes. The two different distributions were noted by many students. Furthermore, the shared memory on the node and distributed across nodes allowed us to discuss the question of data access patterns using concrete examples. Overall, the students developed intuition about shared versus distributed memory, communication and communication latency, load balancing and data distributions, and retained the understanding in the months afterward based on questions fielded by the supercomputing center Computational Science and Engineering Team.

4.4. Virtual adaptation

An essential aspect of the "Be the Processor" game is that it encourages intuition about the hierarchical nature of the HPC architectures and associated communication concerns through the act of executing the instructions of a parallel program. The parallel nature of the game requires that students act cooperatively to complete the task and thus neither a simple simulation, demonstration, nor single-player digital version provides a substitute capable off fulfilling the instructor's goals for the game. Maintaining these aspects was crucial in the virtual adaptation.

During the in-person game, teams were grouped together around a table to mimic shared memory. In this position, they could all reach the "memory registers" and place their character in the correct location. In a virtual environment such as Zoom [21], the use of break-out rooms provides analogous "shared memory nodes" where participants in the break out room are cores and a collaborative tool acts as the shared memory. Local messages can be aggregated in the main room to complete the phrase.

In the first virtual version, students were randomly assigned to break-out rooms with a facilitator to help with the task. Each breakout room was provided with a link to a Google document, shown in Fig. 6b, that included a review of the pseudo-code, the data for each processor and the data locations for the message. The description of the task is shown in Fig. 7. Once a student (processor) had completed their portion of the task, they returned to the main room. In each breakout room there was one lead processor who had the added task of creating a message with the breakout room number (Node number), message length and message text. Once the message was formed it was sent to Process 0, the lead process in the main meeting room via the meeting chat.

As with many conversions from in-person to virtual, there were a few missteps. Beyond the normal chaos associated with using breakout rooms, the fact that Node 0 was also the main lecture room meant students were part of two nodes, confusing the remote nature of distributed memory. Additionally, the "Processor Game" was substituted for the "Puzzle Game" which is played toward the beginning of the course. While students have experience with puzzles and know what to do, they did not have enough theoretical preparation for the "Processor Game". Moving the game later in the course flow would have provided smoother game play and the analogies would have been clearer. We encountered some classic, HPC errors, for example, the permissions on the Google Documents were incorrect and the number of tasks per breakout room were hard-coded based on the expected number of students. When fewer students showed up, many students had to complete

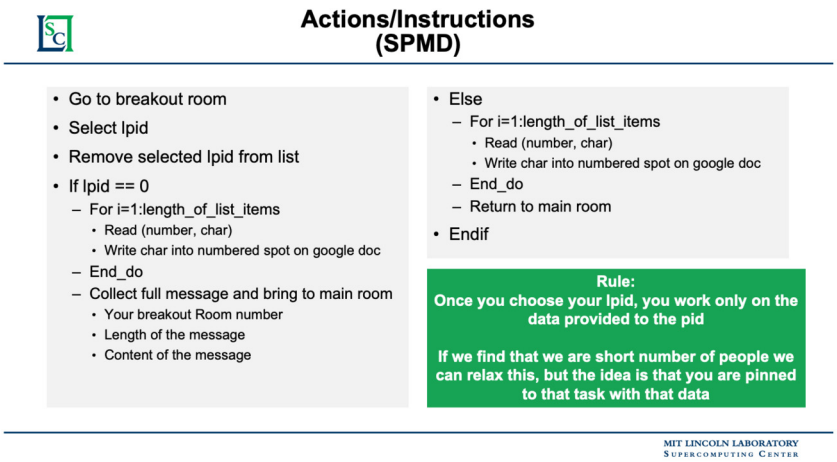


Fig. 7. Psuedo-code for "Be the Processor".

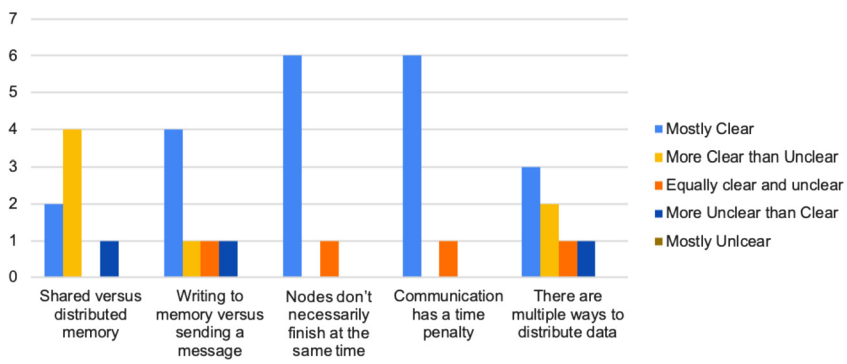


Fig. 8. Survey results for the Be the Processor Game. Students were asked to say how clear the game made the listed topics.

the work of more than one core, which distracted from the distributed computing learning goals. Among next steps for this game is the creation of a bank of phrases, a script that can compute distributions based on number of desired nodes and number of students and a method to pre-populate the collaborative document that serves as the shared memory. These extensions will allow the facilitators to setup the game in real time.

4.5. Evaluation

As this was the first time the virtual game was played it was important to understand whether or not the learning objectives were clearly demonstrated through game play. Student learning was evaluated through a post-game survey. The first question asked to what extent playing the game clarified the learning objectives. The results are shown in Fig. 8. Students recognized that processors operate independently, don't necessary complete in unison, and that communication has a time penalty. The distinction between shared and distributed memory was mostly understood, while the difference between writing to shared memory and sending a message was less clear. Students recognized that they were working on cyclic data in the breakout room but didn't make the connection to block distribution when the messages were collected by Processor 0. The students were also asked how much playing the game helped them understand MPI message passing requirements, the trade-offs between HPC workflows, and the Single Program Multiple Data (SPMD) programming paradigm. The results are displayed in Table 4. As part of the game, each breakout room leader creates a message with data that is required by MPI but facilitators didn't focus on specific syntax at this point in the course which explains the responses. The students understood that each processor was executing the same task on different data, but connecting these concepts to student workflows needs to be revisited. One possibility is to align the pseudo-code they execute to workflows discussed in the course.

Table 4
Average helpfulness of the "Be the Processor" game. Students were asked to rate how much the "Be the Processor" game helped them understand some topics on a scale of 1 to 5. This table shows the average of these scores.

Topic	Average Response
MPI message requirements	3
How to evaluate the trade-offs between workflows	2.71
SPMD	3.57

5. Lessons learned

As stated previously, each of these games has been used in informal, not-for-credit workshops where there were no formal tests or assignments, so student learning was assessed through discussion and surveys. In each instance where the games were used the instructors engaged in post game discussions that provided insight into what the students were taking away from the experience. First and foremost, the students were engaged. They were actively working to understand the concepts around which the games were designed, and they discussed their observations with their team members and asked key questions. During later sections of the course, it was significantly easier to introduce more complex topics because the instructors could refer back to the games, thus scaffolding the new material on top of the experiential learning from the game play. While qualitative, based on the discussions during the workshop and the questions to the MIT Supercloud's Computational Science and Engineering Team, it was clear that the

students had developed intuition and confidence about the content presented through the game experience.

In order to achieve this level of engagement and learning, the instructors needed to modify the games between offerings. Among the lessons that the instructors learned, the following stand out:

- It is important to play a round of the game or demonstrate the task so that students understand what is expected. As mentioned by [10] this lowers the cognitive load.
- The level of work must be appropriate to the team. The instructors needed to adjust the size of the puzzles to match the task and provide the right level of work for both the shared and distributed puzzle games.
- It is helpful to provide students with thought exercises before starting the game. Questions posed as part of the game overview were revisited during discussion and evaluated based on the student experience.
- It is important to have tangible, physical components to help keep students engaged and involved.
- Smaller teams provided for more collaboration which led to more social learning.
- It is important to have enough facilitators so that students are able to have Just-In-Time feedback to their questions and observations.
- The game start-up cannot take a long time or students lose interest. The scheduler game was revised to fill the queue prior to starting game play to overcome this.
- Players may make run-time modifications that you hadn't considered, especially for collaborative games. In the scheduler game one team of students elected a single person to place jobs on the board and the rest decided where they should be placed.
- The games are self-contained and can be used as unique modules but it is important to provide appropriate context and background in order for the students to appreciate the game and develop intuition. This requires considering the course flow when including the game.

In addition to developing intuition and prompting discussion, the games had the added benefit of being an ice-breaker in a short three day workshop comprised of strangers. The Puzzle Game was played early on the first day and the collaborative nature created a safe learning space where everyone felt comfortable asking questions. This comfort level, once created, carried over through the workshop and enabled greater discussion and sharing. While this had been one of the instructor goals in integrating serious games into the workshop, the level of success exceeded expectations.

6. Next steps

Building on the post-game surveys from the January 2021 virtual course, the instructors plan to add short pre-game surveys in order to capture differences in understanding based on the experience of playing the game. The survey questions for the "Be the Scheulder" game captured impressions about the learning objectives for the game and the course, but it is likely that additional questions can provide insight that will lead to improvements in game play and clarity of the learning objectives. Questions posed for the "Be the Processor" game need review, for example the MPI question was appropriate to the course but less so the game. The instructors expect to refine those questions prior to the next course offering. Furthermore, the location of the "Be the Processor" game in the course flow needs to be reconsidered and the instructors expect to re-position it so that students have more distributed computing background prior to playing the game. Another possibility is to modify the pseudo-code to execute the simpler workflows described in the "Be the Scheduler" game as few of the students develop MPI codes and thus have less direct interest in the details of the architecture.

Additionally, the "Be the Scheduler" game is conducive for being converted into a digital game that is played on a computer and provides an opportunity for participants to achieve the learning objectives while being remote. Instructors may also see additional benefits, such as quicker setup time and more flexibility on creating custom scenarios, both of which allow for the creation of larger clusters and larger number of jobs to be included during game play.

A digital version of the "Be the Scheduler" game is currently in development as a web application, using a JavaScript library called D3 [22] for visualizing the components of the game, such as individual nodes and current jobs. Creating a web application will make the game easily accessible by any student who wishes to play. In its current state, instructors can edit an existing CSV configuration file to specify the number and type of nodes in the cluster and define types and sequences of jobs that will be presented to the players during game play. When the game starts, a visual of the cluster is presented, showing each node and the individual cores and GPUs available. Each core or GPU will change color on the screen to indicate that it is busy. For each turn a job from the queue is displayed on the screen and the students will select a node and the number of job tasks they want to run through the user interface, and then submit their selection. As development continues, jobs will have a run time associated with them, and remove themselves from the board when completed. The game will also collect metrics, such as average utilization and average job pending time, so students can implement and compare scheduling strategies.

7. Summary

Overall the experiential learning and engagement resulting from the use of serious games in an HPC education context helped the students develop greater intuition about HPC. The games described in this paper are self-contained and can be readily incorporated into other classroom settings. One can also modify the games to support slightly different or more specific learning goals, for example changing the distribution in "Be the Processor" would lead to a different set of questions, observations, and discussions. Additionally, modifying the pseudo-code provides an opportunity to explore various HPC workflows. The Lego brick format of the "Be the Scheduler Game" allows instructors at other centers to customize the "cluster" to more closely resemble their hardware and job types. To assist other workshops and courses, the authors have created Instructor Guides for each of the games and will be linking them to the SIGHPC Education Chapter Outreach Repository [23].

CRediT authorship contribution statement

Julia Mullen: Conceptualization, Experiment Facilitator, Data Gathering, Writing – original draft preparation, Writing – Reviewing and Editing. **Lauren Milechin:** Conceptualization, Experiment Facilitator, Writing – original draft preparation, Data Gathering and Analysis, Writing – Reviewing and Editing. **Dennis Milechin:** Software, Writing – Original Draft

Declaration of competing interest

The authors of "Teaching and Learning HPC Through Serious Games" do not have any conflicts of interest with regards to the content included in this paper.

References

[1] SIGHPC education chapter, https://sighpceducation.acm.org/resources.html, 2019. (Accessed 28 October 2020).
[2] Shodor and XSEDE, HPC University, 2020, http://hpcuniversity.org/. (Accessed 28 October 2020).
[3] XSEDE, User portal–online training, https://portal.xsede.org/online-training, 2019. (Accessed 28 October 2020).
[4] ARCHER and EPCC and University of Edinburgh, Archer–online training, http://www.archer.ac.uk/training/online/, 2019. (Accessed 28 October 2020).
[5] H. Ebbinghaus, Uber das Gedachtnis: Untersuchungen zur Experimentellen Psychologie, Duncker and Humblot, 1885.
[6] O. McGarry, 5 ways to challenge the forgetting curve, https://www.learnupon.com/blog/ebbinghaus-forgetting-curve/, 2020. (Accessed 28 October 2020).
[7] C.C. Abt, Serious Games, Viking Press, 1970.
[8] M. Ulicsak, M. Wright, Games in education: serious games, https://www.nfer.ac.uk/games-in-education-serious-games, 2010. (Accessed 28 October 2020).
[9] C. Catalano, A. Luccini, M. Mortara, Guidelines for effective design of serious games, Int. J. Serious Games 1 (2014), https://doi.org/10.17083/ijsg.v1i1.8.
[10] A. DeGloria, F. Bellotti, R. Berta, E. Lavagnino, Serious games for education and training, Int. J. Serious Games 1 (2014), https://doi.org/10.17083/ijsg.v1i1.11.
[11] D.A. Kolb, Management and the learning process, Calif. Manag. Rev. 18 (1976) 21–31, https://doi.org/10.2307/41164649.
[12] G. Petri, C.G. von Wangenheim, How games for computing education are evaluated? A systemic literature review, Comput. Educ. 107 (2017) 68–90, https://doi.org/10.1016/j.compedu.2017.01.004.
[13] P.E. Battistella, C.G. von Wangenheim, Games for teaching computing in higher education - a systemic review, IEEE Technol. Eng. Educ. 1 (2016) 8–30.
[14] National Research Council, Division on Engineering and Physical Sciences, The Rise of Games and High-Performance Computing for Modeling and Simulation, National Academies Press, 2010.
[15] ARCHER, EPCC, University of Edinburgh, Supercomputing challenege, github.com/EPCCed/Activities/tree/master/Activities/SuperComputingChallenge, 2017. (Accessed 28 October 2020).
[16] H. Neeman, Supercomputing in plain English, http://www.oscer.ou.edu/education.php, 2018. (Accessed 28 October 2020).
[17] H. Neeman, Supercomputing in plain English: what the heck is supercomputing?, video, https://youtu.be/rB4UI_WODL0?t=3042, 2018.
[18] T. Warburton, Personal Conversation, 2004.
[19] A.B. Yoo, M.A. Jette, M. Grondona, Slurm: simple Linux utility for resource management, in: D. Feitelson, L. Rudolph, U. Schwiegelshohn (Eds.), Job Scheduling Strategies for Parallel Processing, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, pp. 44–60.
[20] C. Byun, J. Kepner, W. Arcand, D. Bestor, B. Bergeron, V. Gadepally, M. Hubbell, P. Michaleas, J. Mullen, A. Prout, A. Rosa, C. Yee, A. Reuther, LLMapReduce: multi-level map-reduce for high performance data analysis, in: 2016 IEEE High Performance Extreme Computing Conference (HPEC), 2016, pp. 1–8.
[21] Zoom, https://zoom.us, 2020. (Accessed 28 October 2020).
[22] D3, https://d3js.org/, 2020. (Accessed 23 February 2021).
[23] SIGHPC Education Chapter, Outreach repository, https://github.com/SIGHPC-Education-Chapter, 2019. (Accessed 28 November 2020).

Julie Mullen is a member of the technical staff in the MIT Lincoln Laboratory Supercomputing Center (LLSC) where she assists researchers in maximizing their use of high performance computing resources in order to minimize their time to solution. As an expert in High Performance Computing for computational engineering applications she focuses on redesigning scientific workflows to streamline processing and improve the performance of computational engineering applications.

Dr. Mullen leads the design and creation of online professional education courseware for the LLSC. As part of this effort she facilitates the development of new tools for the Open edX platform to provide support for online Laboratory courseware. Her research includes learning analytics for adaptive learning design and the integration of hands-on physical construction and experimentation with MOOC technologies. Her work has published in both the scientific computing and educational domains.

Dr. Mullen received her Ph.D in Engineering from Brown University and her B.S.M.E from Worcester Polytechnic Institute.

Lauren Milechin is Research Staff at MIT Earth, Atmospheric, and Planetary Sciences. Her work involves big data, database technology, and machine learning applied to problems in diverse domains. Lauren also facilitates researchers on the MIT SuperCloud supercomputing system. Previously, Lauren worked as Associate Technical Staff at the Lincoln Laboratory Supercomputing Center. Ms. Milechin received an MS degree in industrial mathematics from the University of Massachusetts, Lowell, focusing in computer science applications, such as machine learning and algorithms. She holds a BS degree in mathematical sciences from Worcester Polytechnic Institute, where she explored mathematical modeling of disease and of population dynamics.